

GPT-1 등장 이전의 배경과 문제의식

GPT는 특정 작업에 맞는 라벨 데이터가 부족하다는 근본적인 문제를 해결하기 위해 등장한 모델이다. GPT 이전에는 다음과 같은 모델들이 발전하며 그 토대를 마련하였다.

- **Word2Vec**: 단어를 저차원 벡터로 표현하여 단어 간의 유사성을 포착할 수 있게 했다. 하지만 하나의 단어에 고정된 벡터를 할당하기 때문에 문맥에 따라 의미가 변하는 단어를 구분하지 못하는 한계가 있었다.

<aside> 

단어 하나가 여러 가지 뜻을 가질 수 있는 언어의 특성

예를 들어, 'bank'라는 단어를 생각해보았을 때 이 단어는 다음과 같이 두 가지 다른 의미로 사용될 수 있음.

1. '은행'이라는 의미: "I went to the **bank** to deposit money." (나는 돈을 입금하러 **은행**에 갔다.)
2. '강둑'이라는 의미: "He sat on the river **bank** and fished." (그는 강둑에 앉아 낚시를 했다.)

→ Word2Vec 모델은 이 두 문장의 'bank'를 모두 똑같은 벡터로 표현함.

즉, 문맥을 고려하지 않고 'bank'라는 단어 그 자체에만 하나의 의미를 부여하는 것이기 때문에 모델은 '은행'과 '강둑'이라는 전혀 다른 의미를 구분하지 못하게 되는 것임.

이러한 한계는 같은 단어라도 문맥에 따라 의미가 달라지는 **다의어(polysemy)** 문제를 해결하지 못한다는 것을 의미함. 이후에 나온 ELMo나 GPT 같은 모델들은 이러한 한계를 극복하기 위해, **문맥을 반영하여 단어의 의미를 벡터로 표현**하는 '문맥 기반(Contextual) 임베딩' 방식을 사용하게 됨.

</aside>

- **Seq2Seq + Attention**: 긴 문장의 정보 손실 문제를 해결하기 위해 어텐션 메커니즘을 도입하였다. 이 덕분에 긴 문장 번역 품질이 개선되었으나, 여전히 RNN 기반이어서 병렬 학습이 어렵고 속도가 느리다는 단점이 있었다.

<aside> 

- **RNN의 작동 방식:** RNN은 문장을 처리할 때, "나는" -> "학교에" -> "간다"와 같이 단어를 **하나씩 순차적으로** 읽어들이고, 이전 단어의 정보를 다음 단어에 전달한다. 이 과정은 마치 for 반복문처럼 한 번에 하나의 단계만 처리할 수 있다.
- **병렬 학습의 어려움:** 이 때문에 "학교에"를 처리하려면 "나는"을 먼저 처리해야 하고, "간다"를 처리하려면 "나는 학교에"를 먼저 처리해야 한다. 즉, 앞 단계의 계산이 끝나야만 다음 단계의 계산을 시작할 수 있다. 이처럼 **순차적으로 의존적인 구조** 때문에 여러 개의 계산을 동시에 처리하는 **병렬화**가 불가능하다.
- **느린 속도:** 현대의 GPU는 여러 작업을 동시에 처리(병렬 처리)하는 데 최적화되어 있다. 하지만 RNN은 이런 GPU의 장점을 제대로 활용하지 못하고, 문장의 길이가 길어질수록 계산에 걸리는 시간이 선형적으로 증가하게 된다. </aside>
- **Transformer:** RNN/CNN을 제거하고 셀프 어텐션을 도입하여 모델의 완전 병렬화를 가능하게 하였다. 이를 통해 학습 속도가 수십 배 개선되고 긴 문맥 처리 성능이 향상되었다.

<aside> 

Transformer의 혁신과 한계

- **혁신:** Transformer는 RNN/CNN의 순차적인 처리 방식을 버리고 **병렬 처리**를 가능하게 했다. 덕분에 학습 속도가 빨라졌고, 문맥을 한 번에 파악하여 긴 문장에서도 정보를 잃지 않는 강력한 구조를 갖게 되었다. 즉, "어떻게 하면 문장을 더 효율적으로, 더 똑똑하게 처리할까?"에 대한 해답이었다.
- **한계:** 하지만 Transformer 자체는 모델 구조일 뿐, "어떤 데이터로, 어떤 방식으로 학습시킬까?"라는 문제에 대한 답은 아니었다. 여전히 Transformer 기반 모델을 특정 작업(번역, 요약 등)에 사용하려면 **해당 작업에 맞는 라벨링된 데이터**가 필요했다.

라벨링된 데이터의 필요성

어떤 그릇이든 그 안에 담을 무언가가 필요하듯이, 트랜스포머 모델도 학습을 위한 데이터가 필수적이다. 트랜스포머는 그 자체로 특정 작업을 수행하는 것이 아니라, 특정 작업을 위한 데이터로 학습되어야 그 능력을 발휘할 수 있다. 예를 들어:

- **번역 모델**을 만들려면 "영어 문장 - 한국어 문장" 쌍으로 이루어진 라벨링된 데이터가 필요하다.
- **요약 모델**을 만들려면 "긴 문서 - 짧은 요약문" 쌍으로 이루어진 라벨링된 데이터가 필요하다. </aside>

- **ELMo**: Word2Vec의 정적 임베딩 한계를 극복하기 위해 문맥에 따라 단어의 벡터가 달라지는 문맥 기반 임베딩을 제시하였다. 이 모델은 사전 학습된 언어 모델을 활용한 전이 학습이 효과적임을 입증하였다.

<aside> 

Word2Vec과 같은 이전 모델들은 각 단어에 고정된 하나의 벡터를 할당하는 '정적 임베딩' 방식을 사용했다. 이것은 'bank'라는 단어가 '은행'을 뜻하든 '강둑'을 뜻하든 상관없이 똑같은 벡터를 갖는다는 것을 의미한다. 이는 언어의

다의성을 제대로 반영하지 못하는 심각한 한계였다.

ELMo는 이 문제를 해결하기 위해 '문맥 기반 임베딩(contextualized embedding)'이라는 개념을 도입했다. ELMo 모델은 문장 전체를 입력으로 받아, 같은 단어라도 문맥에 따라 다른 벡터를 생성한다.

예를 들어:

- "I went to the **bank** to deposit money." (나는 돈을 입금하러 **은행**에 갔다.)
- "He sat on the river **bank** and fished." (그는 강 **둑**에 앉아 낚시를 했다.)

ELMo는 이 두 문장의 'bank'를 각각 다른 의미를 가진 벡터로 표현한다. 마치 단어가 문장 속에서 살아있는 것처럼, 주변 단어들을 보며 실시간으로 자신의 의미를 바꾸는 것과 같다.

</aside>

GPT는 이러한 선행 모델들의 구조적 한계와 사전 학습의 효과를 바탕으로, 방대한 양의 **비라벨 텍스트 데이터**를 활용하여 언어 이해 능력을 향상시키는 새로운 접근법을 제시한 것이다.

논문 핵심 아이디어와 모델 구조

GPT의 핵심은 생성 사전 학습과 판별 미세 조정이라는 두 단계 학습 방식에 있다.

핵심 아이디어

- **사전 학습 (Generative Pre-training)**: 대량의 비라벨 텍스트 데이터(BookCorpus)를 사용하여 범용적인 언어 모델을 미리 학습시킨다. 이 단계에서 모델은 언어의 일반적인 구조와 패턴, 그리고 장기적인 의존성을 학습하게 된다.

- **미세 조정 (Discriminative Fine-tuning):** 사전 학습된 모델을 소량의 라벨 데이터를 가진 특정 NLP 태스크에 맞춰 미세 조정한다. 이를 통해 모델은 사전 학습으로 얻은 지식을 바탕으로 특정 작업에 대한 성능을 높일 수 있다.
- **Transformer 디코더 사용:** GPT 는 Transformer 의 디코더 블록만을 활용하여 단방향 언어 모델을 구성하였다. 이는 다음 단어를 예측하는 언어 모델링 목표에 부합하며, 병렬 학습의 이점을 유지하게 해준다.

특징	사전 학습 (Pre-training)	미세 조정 (Fine-tuning)
목표	언어의 일반적인 지식 습득	특정 과제에 대한 전문화
사용 데이터	대규모 비라벨(Unlabeled) 데이터	소규모 라벨링된(Labeled) 데이터
학습 방법	비지도 학습	지도 학습
결과	범용적인 언어 모델 구축	특정 과제에 최적화된 모델 구축
비유	여러 책을 읽으며 지식을 쌓는 것	그 지식을 활용해 특정 시험을 준비하는 것

모델 구조

GPT-1 은 12 개의 Transformer 디코더 레이어를 쌓아 올린 구조이다.

GPT-1 은 복잡한 뇌를 여러 층으로 쌓아 올린 구조인데, 이 뇌의 기본 단위는 **트랜스포머 디코더**라는 블록이다. 이 블록이 GPT 의 모든 지능적인 연산을 담당한다.

왜 하필 디코더만 사용했을까?

트랜스포머는 원래 문장을 입력받아(인코더) 다른 언어로 번역(디코더)하는

번역 모델을 위해 만들어졌다. 그런데 GPT 의 목표는 번역이 아니라, 문맥을 보고

다음 단어를 예측하는 것이다.

- **인코더:** 문장 전체를 한 번에 보고 정보를 파악한다. (예: "나는 학교에 간다"를 볼 때, '나는'이 '간다'와 관련 있다는 걸 동시에 본다.)
- **디코더:** 이전 단어들을 바탕으로 다음 단어를 하나씩 생성한다. (예: "나는 학교에"를 보고, 다음에 올 단어는 '간다'일 것이라고 예측한다.)

GPT 는 **언어 모델링**이라는 목표에 맞춰, 다음 단어를 예측하는 데 특화된 **디코더만** 사용한 것이다.

이 덕분에 문장을 순서대로 학습하며 병렬 처리의 이점도 놓치지 않았다.

- **입력 임베딩:** 입력 토큰에 위치 임베딩을 더해 입력 벡터를 구성한다.
- **마스크드 멀티-헤드 어텐션:** 단방향 언어 모델의 특성상 미래의 토큰을 참조하지 못하도록 **마스킹**을 적용한 어텐션 메커니즘을 사용한다.

→ 모델이 문장의 **미래 단어를 보지 못하게** 가리는 역할을 한다. "나는 학교에 간다"를 예측할 때, 모델은 "나는 학교에"까지만 보고 "간다"를 예측해야 한다. '마스크'는 이 규칙을 강제하는 장치다.

- **잔차 연결 및 레이어 정규화:** 모델의 깊이가 깊어져도 안정적으로 학습되도록 돕는다.

→ 모델이 12 층이나 쌓여도 학습이 안정적으로 진행되도록 돕는 역할을 한다. 층이 깊어지면 정보가 손실되거나 왜곡될 수 있는데, 이 기술들은 마치 층층마다 엘리베이터를 설치해 정보가 잘 전달되게 하는 것과 같다.

- **잔차 연결(Residual Connections):** 이는 마치 층마다 지름길(shortcut)을 만들어주는 것과 같다. 이전 층의 정보를 다음 층으로 직접 전달함으로써, 정보가 손실되거나 왜곡되는 것을 막아준다.
- **레이어 정규화(Layer Normalization):** 각 층의 출력을 일정한 범위로 조정해 학습을 안정적으로 만들어준다. 이 덕분에 학습이 더 빠르게 수렴하고 성능도 좋아진다.
- **FFN (Feed-Forward Network):** 각 토큰의 표현을 독립적으로 처리하는 신경망 레이어이다.

어텐션이 단어들 간의 관계를 파악하는 '대화'라면, FFN 은 그 대화 내용을 바탕으로 각 단어의 의미를 더 깊이 '생각'하는 과정과 유사하다. 이 네트워크는 두 개의 선형 변환 레이어와 비선형 함수(GPT-1에서는 GELU)로 구성되어 있다.

FFN 은 어텐션 레이어가 단어들 간의 관계를 파악한 후, 그 정보를 바탕으로 **각 단어의 의미를 개별적으로 더 깊게 파고드는 역할**을 한다.

이것을 '심층적으로 정리하고 강화한다'는 건, FFN 이 단순히 정보를 전달하는 게 아니라, 두 개의 선형 변환 레이어와 비선형 함수를 거치며 **단어 표현을 더욱 복잡하고 정교하게 만든다**는 의미다.

어텐션 레이어가 문맥을 고려해 단어의 가중치를 정해줬다면, FFN 은 그 가중치를 바탕으로 단어의 의미를 다차원적으로 확장하고 압축하는 과정을 거친다. 이를 통해 모델은 단어의 표현을 더 풍부하고 강력하게 만들어 다음 단계로 넘겨줄 수 있다.

- **입력 임베딩 (Input Embedding)**

입력 임베딩은 모델이 문장을 이해하기 위한 첫 단계로, 단어에 의미와 순서 정보를 부여하는 과정이다. 모델은 숫자로 된 데이터만 이해할 수 있기 때문에, 문장을 그대로 입력할 수 없고 이를 숫자로 변환해야 한다. 이 과정은 두 가지 주요 구성 요소로 나뉜다.

- **토큰 임베딩 (Token Embedding):** 문장의 각 단어를 고유한 벡터(숫자의 나열)로 변환한다. 이 벡터는 단어의 의미를 담고 있으며, 비슷한 의미의 단어들은 비슷한 벡터 값을 가지게 된다. 예를 들어, '개'와 '강아지'는 비슷한 벡터 값을 가질 수 있다.
- **위치 임베딩 (Positional Embedding):** 문장의 단어들은 순서가 중요하다. "I am a student"와 "A student am I"는 단어는 같지만 의미는 완전히 다르다. RNN 과 달리 트랜스포머는 문장 전체를 한꺼번에 처리하기 때문에 단어의 순서를 잃어버릴 수 있다. 위치 임베딩은 각 단어가 문장 내에서 어디에 위치하는지 알려주는 '주소표' 역할을 한다. 이 '주소표'를 토큰 임베딩에 더함으로써, 모델은 단어의 의미뿐 아니라 순서 정보까지 동시에 파악할 수 있다.

주요 수식과 원리

GPT 의 학습은 두 단계의 목표 함수를 최적화하는 과정이다.

1. **비지도 사전 학습:** 비라벨 텍스트 코퍼스에 대해 다음 토큰이 나타날 확률을 최대화한다.

→ 최종 은닉 상태를 통해 다음 토큰에 대한 확률을 계산한다.

2. **지도 미세 조정:** 라벨링된 데이터셋에 대해 올바른 라벨을 예측하는 확률을 최대화한다.

미세 조정 단계에서는 사전 학습 목표인 L1 을 보조 목표로 함께 사용하여 모델의 일반화 능력을 높이고 수렴 속도를 가속화한다.

이때, 추가되는 파라미터는 출력 레이어의 W_y 와 구분자 토큰 임베딩뿐이다.

모델의 한계 및 후속 연구

GPT-1 은 NLP 분야에 큰 영향을 미쳤지만, 몇 가지 한계점이 있었다.

- **모델 규모의 한계:** 12 개의 레이어와 512 토큰의 컨텍스트 윈도우 크기를 가졌다.
- **데이터셋의 한정성:** 사전 학습에 사용된 데이터셋이 BookCorpus 로 한정되었다.
- **미세 조정의 필요성:** 사전 학습은 비지도 방식으로 이루어졌지만, 실제 작업을 위해서는 라벨링 데이터가 필요한 지도 미세 조정이 필수적이었다.

이러한 한계는 GPT-2 와 GPT-3 로 이어지는 후속 연구의 원동력이 되었다.

- **GPT-2:** 미세 조정 없이도 다양한 태스크를 수행하는 **제로샷(zero-shot)** 성능 향상에 집중하였다. 이를 위해 모델 크기(1.5B 매개변수)와 데이터셋(WebText, 40GB)의 규모를 대폭 확장하였다.
- **GPT-3:** 모델 규모를 더욱 키우고(175B 매개변수) 더 방대한 데이터(570GB)를 사용하여 **인-컨텍스트 러닝(in-context learning)** 능력을 발전시켰다. 이 덕분에 별도의 미세 조정 없이도 몇 가지 예시만으로 작업을 잘 수행할 수 있게 되었다.

이러한 발전은 GPT-1 이 제시한 사전 학습-미세 조정 패러다임이 얼마나 강력하고 확장성이 높은지를 입증해준다.